FinSuite

A Web Platform for Financial Analysis, Live Market Data and Next-Day Price Forecasting

Purva Kakade

Research Intern, Indian Institute of Technology Bombay

Under the guidance of **Prof. Ashish Singh**

Shailesh J. Mehta School of Management, IIT Bombay

System documentation and evaluation, version 1.0, October 2026

Abstract

FinSuite is a free website that gathers seventeen finance tools in one place: ratio calculators, loan and credit checks, goal planning, live stock quotes and news, and a next-day price forecast. This document describes how the system is built, where its data originates, how each result is computed, and how the results were checked for correctness. It also reports a small forecasting experiment in which a Linear Regression model and a Random Forest were compared with a naive baseline on four instruments. Neither model improved on the baseline in a consistent way. The deployed site reports this to its users in the forecasting tab.

Keywords: personal finance tools; open data; stock market data; price forecasting; Random Forest; system architecture; software verification

1. Introduction

1.1 Background and problem

Decisions about loans, savings and shares are often made without the analytical tools that professionals use. Many existing tools are paid, require an account or an API key, or return a number without showing how it was obtained. For a beginner it is then hard to judge whether a result is right, or what it means.

1.2 Objectives

The system was designed around five objectives. First, it should be free to run, so all live data comes from public sources that need no registration. Second, every result should be explainable: each tool states its formula, and a built-in Knowledge Center defines each term with a worked example. Third, calculations should be correct, which we checked against values computed by hand, including boundary cases. Fourth, the system should state its limits where a result is weak, as it does for price forecasts. Fifth, it should keep no personal data: there are no accounts, no database, and nothing typed by a user is stored on the server.

1.3 Contributions

This document contributes (i) a description of the architecture and the live-data pipeline, (ii) the formulas and decision rules behind every score and recommendation, (iii) a forecasting experiment comparing two models with a naive baseline, (iv) a record of verification, including the defects found and corrected, and (v) instructions for running and deploying the system again.

2. System Overview

The website has seventeen modules, each on its own tab. We group them by the kind of processing they perform: *calculation* (formulas applied to numbers typed by the user), *rule-based* (fixed scoring rules), *live data* (Yahoo Finance and Google News), *reference* (static text) and *ML* (machine learning). Table 1 lists them.

Table 1. The seventeen modules of FinSuite.

No.	Module	Function	Kind
01	Ratios	ROE, ROA and debt ratio, with a health label.	Calculation
02	Finance Report	Income, expenses, savings and debt converted into ratios and short tips.	Calculation
03	AI Assistant	Answers finance-term questions from stored definitions and gives a financial snapshot. It does not call an external AI service.	Rule-based
04	Loan Insights	Comments on the credit profile of each customer in a loan table.	Rule-based
05	Credit and Risk	Credit-score category and risk level, adjusted for the EMI burden.	Rule-based
06	Investment Call	BUY, HOLD or AVOID from four threshold checks.	Rule-based
07	Company Performance	Score out of 100 from growth, customer satisfaction and staff retention.	Rule-based
08	Portfolio Dashboard	Scores and ranks several companies.	Rule-based
09	Goals and Retirement	Monthly saving needed for a goal, with inflation; emergency-fund check.	Calculation
10	Knowledge Center	Sixty-seven plain-language entries with examples, in six groups, with search.	Reference
11	Stock Market	Search any listed company; live quote, daily history table and chart, news.	Live data
12	Growth and Target	Present value of an earlier investment; target and stop-loss planning.	Calculation
13	Portfolio and Tax	SIP versus lump sum, capital-gains tax estimate, 52-week range position.	Calculation
14	Watchlist and Alerts	Live prices for chosen stocks and price alerts.	Live data
15	Data Sources	States where the information comes from.	Reference
16	About	Project, designer and guide.	Reference
17	ML Prediction	Next-day close forecast from two models, shown beside a baseline.	ML

3. System Architecture

FinSuite follows a conventional layered design (Figure 1). The browser renders pages and sends requests. The application server receives each request and validates its inputs. A services layer performs the actual work, namely the formulas, the market-data handling and the forecasting. Only the market module contacts outside sources, and it uses two public ones.

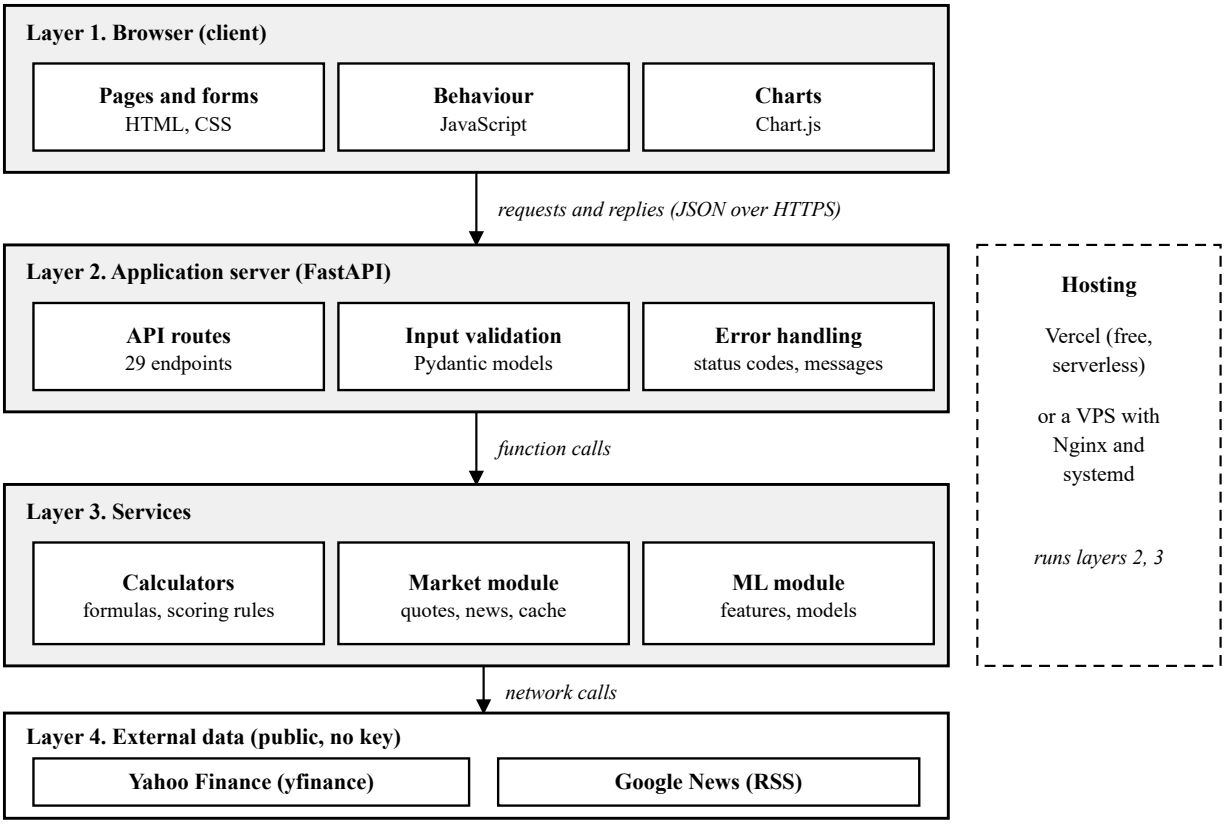


Figure 1. System architecture. Requests pass down through the layers and replies return the same way.

3.1 Layers

The client is plain HTML, CSS and JavaScript, with the Chart.js library for charts; no front-end framework is used, which keeps the code short and readable. The server is written in Python with FastAPI and Uvicorn. It exposes 29 endpoints, each a web address that performs one task, validates its input with Pydantic models and converts failures into clear messages instead of server errors. The services layer consists of three Python modules: the calculators, the market module (search, quotes, history, news and an in-memory cache) and the ML module (features and models). The external data layer consists of Yahoo Finance, accessed through the `yfinance` library, and the public Google News RSS feed.

3.2 Design decisions

Table 2. Principal design decisions and their trade-offs.

Decision	Rationale	Trade-off
Python and FastAPI	Financial mathematics and machine learning share one language; inputs are validated automatically.	Requires a Python runtime for hosting.
No front-end framework	Small code base, no build step, easy to read.	More manual code in the page.
No database or login	No personal data held; simple and private.	The watchlist and alerts are lost when the page is reloaded.
In-memory cache	Protects the free data sources and speeds up repeated requests.	Cleared whenever the server restarts.

Decision	Rationale	Trade-off
Public data, no keys	Anyone can reproduce the system at no cost.	The sources are unofficial and may be delayed or limited.
Version stamp on script and style files	A browser never pairs a new page with an old script after an update.	None of note.
Discard out-of-date replies	If a second stock is chosen quickly, the late reply for the first is ignored.	A little extra client code.

4. Data Collection

4.1 Sources

Table 3. Data sources and how they are read.

Source	Data	Access	Used by
Yahoo Finance	Company search, latest price, daily open, high, low, close and volume	`yfinance` library, no key	Modules 11, 14, 17
Google News	Headlines from the last seven days	Public RSS feed, no key	Module 11
The user	Numbers typed into forms	Sent to the server, not stored	All other modules

4.2 Retrieval of a live quote

Figure 2 shows what happens when a quote is requested. The server first asks the cache for a stored answer; if none is younger than 30 seconds it asks Yahoo Finance, stores the result and replies. To obtain the last traded price, the server reads daily bars to find the previous close and then reads the latest one-minute bars to find the last trade and its time. On weekends and holidays the most recent session is shown together with its time, so that a user can tell the market is closed.

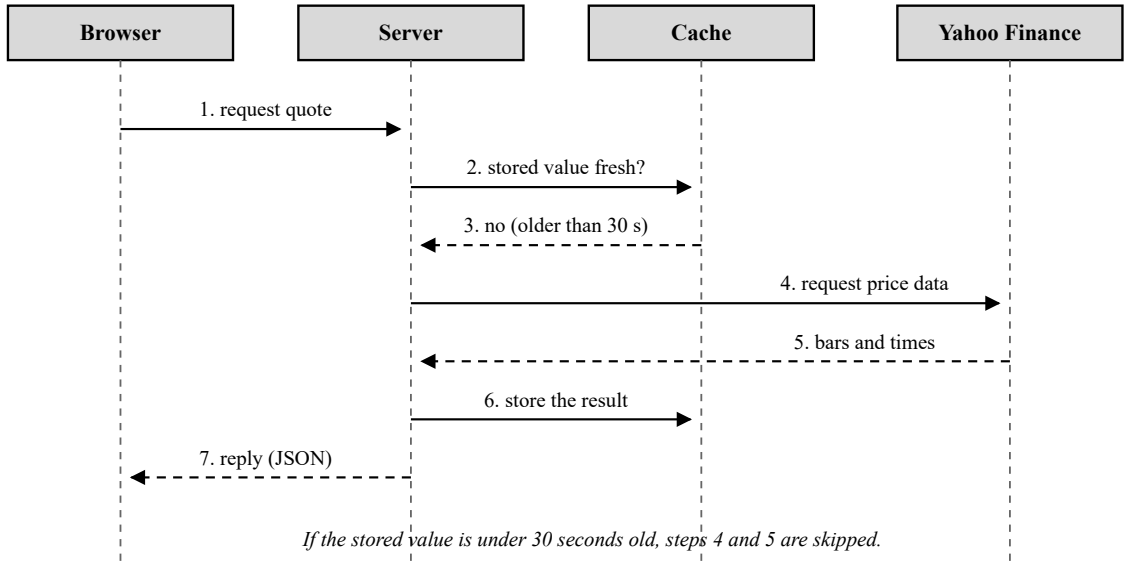


Figure 2. Sequence of messages for one live quote.

4.3 Symbol resolution

A user may enter a company name (for example "Tata Steel") or a code ("TATASTEEL.NS"). Names are sent to Yahoo's search first. Codes are tried as typed, then with the suffix `.NS` (National Stock Exchange)

and then `.BO` (Bombay Stock Exchange). The first candidate that returns price data is used; if none does, a clear message is shown (Figure 3).

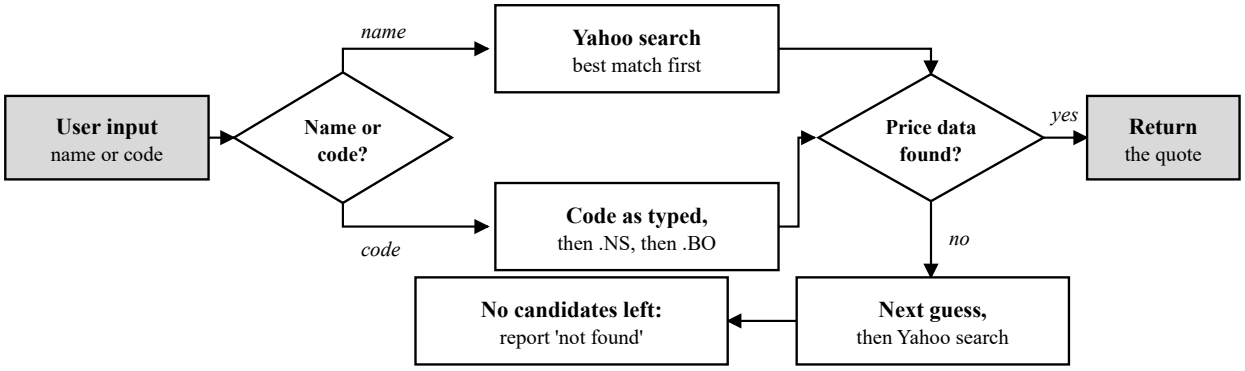


Figure 3. Conversion of user input into a symbol that has data.

4.4 Caching and fault tolerance

Table 4. Retention time of cached data.

Data	Retained for	Reason
Live price	30 seconds	Prices change often, but repeated clicks need no new request.
Daily history	10 minutes	Daily bars rarely change within a day.
News	5 minutes	Headlines change slowly.
Company search	1 hour	Company names are stable.

If a source cannot be reached, the last successful answer is returned instead of an error. Automatic refresh runs only while the relevant tab is open and visible: quotes every 30 seconds and news every 5 minutes.

4.5 News relevance

A plain text search for a company name returns many pages that merely mention it. A headline is therefore kept only if its title contains the company name or its ticker. Duplicate copies of a story are removed and the newest headlines are shown first.

5. Methods

5.1 Calculation tools

Table 5. Formulas and interpretation rules used by the calculation tools.

Quantity	Formula	Interpretation rule
Return on equity (ROE)	*Net income / Equity × 100*	Above 20% excellent; 10 to 20% good; below 10% weak
Return on assets (ROA)	*Net income / Assets × 100*	10% or more treated as strong
Debt ratio	*Liabilities / Assets*	Higher means more borrowing
Debt to equity	*Debt / Equity*	Above 1: owes more than owns
Current ratio (personal)	*Savings / Monthly expenses*	Months of expenses covered

Quantity	Formula	Interpretation rule
Savings ratio	*Savings / Income × 100*	Below 20% flagged as thin
Risk-reward ratio	*(Target − Entry) / (Entry − Stop)*	2 or more favourable; 1 to 2 modest; below 1 weak
Compound annual growth rate	$(Value\ now / Amount)^{1/years} - 1$	Shown for holding periods of about a month or more
52-week position	*(Price − Low) / (High − Low)*	0% at the low, 100% at the high
Emergency fund	*Savings / Monthly expenses*	6 months or more protected; 3 to 6 building; under 3 critical

5.2 Goal planning

A target stated in today's money is first raised by inflation, so the future target is $Target \times (1 + inflation)^{years}$. Existing savings are grown at the expected monthly return, and the monthly investment is the amount that closes the remaining gap. With monthly rate i equal to the annual return divided by 12 and n the number of months, the monthly amount is

$$SIP = Remaining\ target \,/\, [\ ((1 + i)^n - 1) / i \times (1 + i)\ ]$$

Savings and SIP use the same monthly compounding, so the yearly schedule ends exactly on the target. An earlier version mixed yearly and monthly compounding and failed this check (Section 6).

5.3 Capital-gains tax

The tool follows the Indian rules for listed shares as modelled here. A holding of 365 days or fewer is short term and its gain is taxed at 20%. A longer holding is long term, taxed at 12.5% on the part of the year's gains above Rs 1,25,000. A 4% cess is added to the tax, and a loss attracts no tax. The rules change over time, so the output is an estimate and not tax advice.

5.4 Rule-based scores

The scores in Table 6 are expert rules with conventional thresholds. They were not fitted to data; their purpose is to teach, and each rule is displayed to the user.

Table 6. Decision rules behind the rule-based modules.

Score	Rule	Result
Credit category	Score bands with limits at 550, 650, 750 and 800	Poor, Fair, Good, Very Good, Excellent
Credit risk level	Base level from the band; raised one step if the EMI exceeds 40% of income	Very High to Very Low risk
Investment call	One point each for revenue growth of 15% or more, profit margin of 15% or more, ROA of 10% or more, and ROE of 15% or more	3 or 4 BUY; 2 HOLD; below 2 AVOID
Portfolio score	ROE above 15% (30 points), ROA above 10% (25), growth above 10% (25), margin above 15% (20); scaled to the figures supplied	75 or more BUY; 50 to 74 HOLD; below 50 SELL
Company performance	Growth above 15% (34), satisfaction above 80% (33), retention above 85% (33)	85 or more excellent; 60 or more good; otherwise needs improvement

5.5 Machine-learning forecast

The ML module forecasts the closing price of the next trading day for a chosen symbol, using only that symbol's own price history (Figure 4). Between 1 and 5 years of daily bars are downloaded, and at least 120 trading days are required. Rows with a missing close and repeated dates are removed. Five inputs are then computed for each day: the close, the mean close of the last 7 and of the last 30 days, the percentage change from the previous day, and volatility, defined as the standard deviation of daily changes over the last 21 days. The target is the close of the following day.

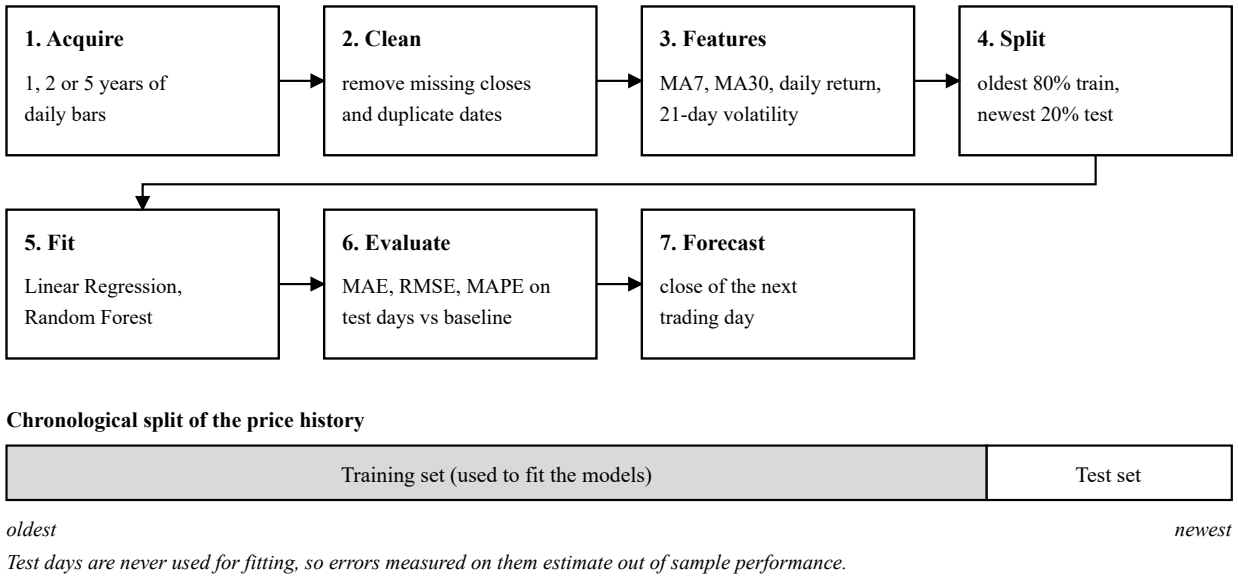


Figure 4. Forecasting pipeline and chronological split.

The data are split by time and never shuffled: the oldest 80% of the days are used for fitting and the newest 20% for testing. Shuffling would allow a model to see days after the ones it is asked to predict and would overstate its accuracy. Two models are fitted with scikit-learn (Pedregosa et al., 2011): a Linear Regression on standardised inputs, and a Random Forest (Breiman, 2001) with 200 trees and a maximum depth of 8. The forest predicts the change from the current close and adds it back. Both are compared with a naive baseline that forecasts tomorrow's close as today's. Errors are measured as the mean absolute error (MAE), the root mean squared error (RMSE) and the mean absolute percentage error (MAPE) (Hyndman and Koehler, 2006); lower values are better.

The choice of target for the forest was changed during development. A first version predicted the price level directly and performed poorly: on TCS its MAE was 112.16 against 35.21 for the baseline. A tree-based model cannot return a value outside the range of prices seen in training, and prices tend to drift upward. Predicting the day's change instead reduced the MAE to 47.30. The weakness was therefore in the model's structure, not in the data.

6. Verification

6.1 Approach

Verification used five kinds of check. For each calculator the expected answer was worked out independently and compared with the output, including values on either side of every threshold (for example credit scores of 549 and 550, or 749 and 750). Invalid inputs such as zero divisors, out-of-range values and impossible dates must produce a clear message and never a server error. Live data were checked for internal consistency: the price change must equal the price minus the previous close, the

price must lie within the day's range, and each day's high must not be below its open or close. The last point of the goal schedule must equal the inflation-adjusted target. Finally, all seventeen tabs were exercised in a real browser against the deployed site, including error messages and a phone-sized screen. More than 100 automated checks were run and repeated after each change. For one instrument (Apple), the reported forecast errors were also recomputed independently from the returned predictions and matched.

6.2 Defects found and corrected

Table 7. Defects found during verification and their corrections.

Area	Defect	Correction
Goal planner	With existing savings, savings grew yearly while the SIP grew monthly. At year 10 the plan showed Rs 18,29,755 against a target of Rs 17,90,848.	Both use monthly compounding; the schedule now ends on the target.
Goal planner	A 2.5-year plan stopped at year 2 (Rs 8,97,038 against Rs 11,56,817).	The schedule ends at the true end of the plan.
52-week range	A price below the low showed 2% instead of 0%.	Position limited to 0 to 100%.
Assistant lookup	"Debt to equity" returned the definition of "equity"; "premium" matched "EMI".	Whole-word matching, longest phrase first.
SIP and range tools	A zero price caused a server error.	A validation message is returned.
Input validation	A negative current price gave a growth of −105%; a stop loss above 100% gave a negative stop price; a risk score of 1000 was accepted.	Such inputs are rejected with a message.
ML model	The Random Forest could not extrapolate (Section 5.5).	It now predicts the change from the current close.
Minor	A trailing comma in the SIP price list; buttons that failed silently; a 404 for the site icon; "15.0 year(s)" in a message.	Each corrected.

7. Results of the Forecasting Experiment

Each model was trained on two years of daily history, with the oldest 80% used for fitting and the newest 20%, about 93 trading days, for testing. The last trading day in the data was 16 October 2026. Four instruments were used: TCS and Reliance (Indian equities), Apple (a US equity) and the NIFTY 50 index. Table 8 gives the MAE on the test days and Figure 5 shows each model's error relative to the baseline.

Table 8. Mean absolute error on the test days (lower is better).

Instrument	Baseline MAE	Linear Regression MAE	Random Forest MAE	Baseline MAPE
TCS (Rs)	35.21	35.67	47.30	1.58%
Reliance (Rs)	13.02	13.08	13.25	1.02%
Apple (USD)	3.97	3.99	4.03	1.27%
NIFTY 50 (points)	134.97	132.04	140.52	0.57%

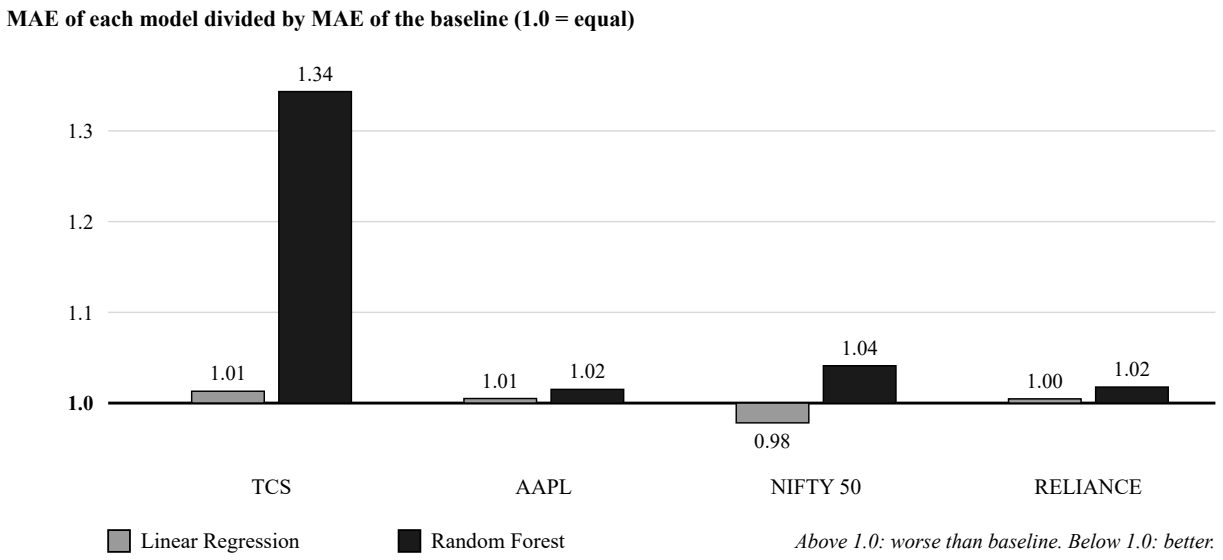


Figure 5. Model MAE divided by baseline MAE for each instrument.

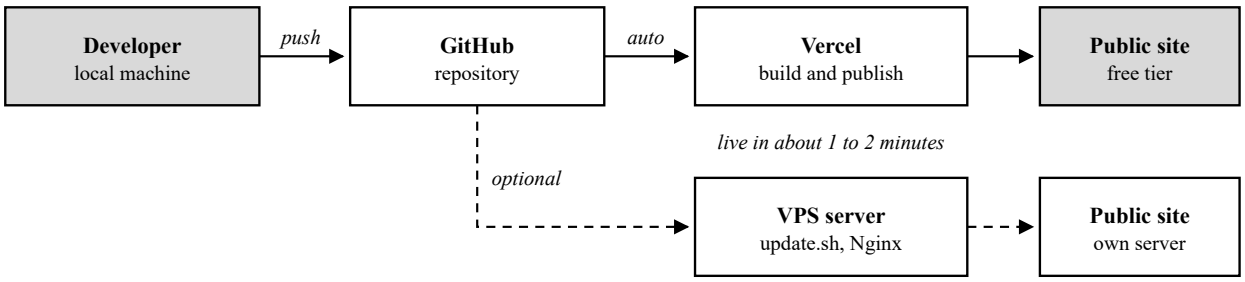
7.1 Discussion

In seven of the eight model and instrument pairs the model was slightly worse than the naive forecast. The exception was Linear Regression on the NIFTY 50, which was about 2% better. Six of the eight differences are within 2%, which is small for a single test window; the larger gaps were the Random Forest on TCS (34% worse) and on the NIFTY 50 (4% worse). These results are consistent with the weak form of the efficient-market hypothesis (Fama, 1970), under which past prices carry little information about the next day's price.

The experiment has clear limits. There is one test window of about 93 days per instrument and a single run, with no significance test or confidence interval. Only four instruments were used, trading costs were ignored, and the models saw price history only, without news, earnings or market-wide data. The results show that these two simple models add little over the baseline; they do not show that no model could.

8. Deployment and Reproducibility

The public site runs on the free plan of Vercel. Each time the code on GitHub changes, Vercel rebuilds and publishes the site, which takes one to two minutes (Figure 6). The free plan is meant for personal, non-commercial use, which suits a research project. The ML tab runs on this plan; a forecast took about four seconds in our check. For a dedicated server, the repository includes files for an Ubuntu VPS in the `deploy` folder: a service definition, an Nginx configuration and setup and update scripts. These were not used for the public site.



Solid: path in use. Dashed: supported by files in the deploy folder, not used for the public site.

Figure 6. Path from the developer's machine to the public site.

8.1 Running the system locally

Python 3.11 or newer is required (3.13 was used), together with an internet connection for live data. The commands below create an isolated environment, install the dependencies and start the server, which then answers at `http://127.0.0.1:8000`. The `PORT` environment variable selects another port.

```
python -m venv agentic_env
agentic_env\Scripts\activate        (Mac or Linux: source agentic_env/bin/activate)
pip install -r requirements.txt
python app/main.py
```

8.2 Project layout

Table 9. Organisation of the repository.

Location	Contents
frontend/	Page (index.html), behaviour (app.js) and styling (style.css).
app/main.py	Server: all endpoints, error handling, serving of the page.
app/calculators.py	Formulas and rules for modules 01 to 09, 12 and 13.
app/market.py	Search, symbol resolution, quotes, history, news and the cache.
app/predict.py, backend/services/	Forecast pipeline: cleaning, features and the two models.
app/glossary.py	The sixty-seven Knowledge Center entries.
deploy/, docs/	Optional VPS files and guide; project report and this document.

9. Limitations

Yahoo Finance data are unofficial, may be delayed and may be rate-limited from some servers; the availability of free public sources cannot be guaranteed. The thresholds used in the scoring rules, such as a 15% ROE or an EMI above 40% of income, are common rules of thumb and were not calibrated on data. The tax tool covers only Indian listed shares and simplifies the rules. The watchlist and alerts live in the browser tab, and alerts work only while the page is open. The assistant matches keywords to stored definitions and does not interpret free text. The forecasting experiment is narrow (Section 7), and its output should not be used for trading. Finally, we verified correctness but did not study how easy the tools are for real users.

10. Future Work

Four directions follow from this work. The first is to make the site more complete as a product: a saved watchlist and alerts, user accounts, background alerts by email, and PDF or Excel export of results. The second is to add tools: a portfolio tracker with live profit and loss, an income-tax comparison of the old and new regimes, an EMI schedule with prepayment, a mutual-fund tracker using public NAV data, a budget planner and a retirement calculator. The third is richer market analysis: technical indicators such as RSI, moving averages and Bollinger bands, news sentiment linked to prices, and stock comparison. The fourth is stronger evaluation of the forecasts, with walk-forward testing over many windows, more instruments, significance tests, additional inputs such as market-wide indicators, and a usability study.

11. Glossary

Table 10. Technical terms used in this document.

Term	Meaning
Endpoint	A web address on the server that performs one task, for example calculating ROE.
REST, JSON	A common way for programs to exchange requests and replies as structured text.
Cache	Short-term storage that avoids asking a source for the same data twice.
RSS feed	A public list of the latest headlines that programs can read.
yfinance	An open-source library that reads public Yahoo Finance data.
Baseline	The simplest forecast that a model must beat to be useful.
Serverless	Hosting in which the provider starts the program only when a request arrives.
VPS	A rented virtual server that the user manages.

References

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

Fama, E. F. (1970). Efficient capital markets: a review of theory and empirical work. *The Journal of Finance*, 25(2), 383–417.

Hyndman, R. J., and Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4), 679–688.

Pedregosa, F., et al. (2011). Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Availability and contact

Live system: https://finsuite-py-6ol4.vercel.app/
Source code: https://github.com/purvakakade450/finsuite-py

Author: Purva Kakade, Research Intern, IIT Bombay. Phone 8975882235. Email purvakakade450@gmail.com. LinkedIn: linkedin.com/in/purva-kakade-356a59237
Guide: Prof. Ashish Singh, SJMSOM, IIT Bombay (https://www.som.iitb.ac.in/prof-ashish-singh/)

FinSuite is an educational tool. Nothing in this document or on the website is financial, tax or investment advice.